

HTL Leonding
Höhere Technische Bundeslehr- und Versuchsanstalt Leonding
Abteilung Informatik

Diplomarbeit

DoggoGO

Eine standortbasierte Web- und iOS-App für Hundehalter

Kandidat: Daniel Hoheneder

Betreuer: Stöttinger Robert

Schuljahr: 2026/27

Abgabedatum: 18.09.2026

Eingereicht an der HTL Leonding zur Erlangung des
akademischen Grades *Ingenieur/in*

Eidesstattliche Erklärung

Ich erkläre an Eides statt, dass ich die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt und die den benutzten Quellen wörtlich oder inhaltlich entnommenen Stellen als solche kenntlich gemacht habe.

Leonding, am 18.09.2026

Daniel Hoheneder

Kurzfassung

Die vorliegende Diplomarbeit beschreibt die Entwicklung von **DoggoGO**, einer standortbasierten Plattform für Hundehalterinnen und Hundehalter. Das Ziel der Anwendung ist es, relevante Informationen – wie die Positionen von Freilaufwiesen, Sackerlspendern und Hundekot-Müllbehältern sowie rassenspezifische gesetzliche Vorschriften (Leinen- und Maulkorbpflicht) – in Echtzeit und standortgebunden bereitzustellen.

Dieser Teil der Arbeit behandelt die technische Umsetzung des **Backends** auf Basis von **ASP.NET Core 8** sowie die Entwicklung der **iOS-Applikation** mit **Swift** und **SwiftUI**. Das Backend stellt eine RESTful API bereit, die von der Angular-Webapplikation (von Manuel Rankl) und der iOS-App konsumiert wird. Als Persistenzschicht kommt **PostgreSQL** in Kombination mit **Entity Framework Core 9** zum Einsatz. Die gesamte Serverinfrastruktur wird mittels **Docker** containerisiert und betrieben.

Die iOS-App ermöglicht die Kartenansicht mit standortbasierten Markierungen über **MapKit**, die Verwaltung von Hundeprofilen sowie die Anzeige rassenspezifischer Rechtsvorschriften. Die Authentifizierung erfolgt über **JSON Web Tokens (JWT)**.

Schlagwörter: ASP.NET Core, Entity Framework Core, PostgreSQL, Docker, iOS, SwiftUI, MapKit, JWT, REST API, Geofencing

Abstract

This diploma thesis describes the development of **DoggoGO**, a location-based platform for dog owners. The application aims to provide relevant information in real time – such as the locations of off-leash areas, poop-bag dispensers, and dog waste bins, as well as breed-specific legal regulations (leash and muzzle requirements) – based on the user’s current location.

This part of the thesis covers the technical implementation of the **backend** using **ASP.NET Core 8** and the development of the **iOS application** using **Swift** and **SwiftUI**. The backend exposes a RESTful API consumed by both the Angular web application (developed by Manuel Rankl) and the iOS app. **PostgreSQL** together with **Entity Framework Core 9** serves as the persistence layer. The entire server infrastructure is containerised and operated using **Docker**.

The iOS app provides a map view with location-based markers via **MapKit**, dog profile management, and the display of breed-specific legal requirements. Authentication is handled via **JSON Web Tokens (JWT)**.

Keywords: ASP.NET Core, Entity Framework Core, PostgreSQL, Docker, iOS, SwiftUI, MapKit, JWT, REST API, Geofencing

Inhaltsverzeichnis

Eidesstattliche Erklärung	1
Kurzfassung	2
Abstract	3
Abbildungsverzeichnis	8
Tabellenverzeichnis	9
Quellcodeverzeichnis	10
1 Einleitung	11
1.1 Ausgangssituation und Problemstellung	11
1.1.1 Defizite bestehender Lösungen	11
1.2 Zielsetzung dieser Arbeit	12
1.3 Abgrenzung	13
1.4 Aufbau der Arbeit	13
2 Anforderungsanalyse	15
2.1 Stakeholder und Zielgruppe	15
2.2 Funktionale Anforderungen	16
2.2.1 Backend-API	16
2.2.2 iOS-Applikation	17
2.3 Nicht-funktionale Anforderungen	18
2.4 Use-Case-Szenarien	18
3 Systemarchitektur	20
3.1 Gesamtarchitektur	20
3.2 Schichten des Backends	21
3.3 Datenbankdesign	21

3.3.1	Beziehungen und Löschverhalten	22
3.4	REST-API-Überblick	22
3.5	Kommunikationsfluss	23
3.6	Sicherheitsarchitektur	24
3.7	Deployment-Architektur	25
4	Technologieauswahl	26
4.1	Backend-Framework: ASP.NET Core 8	26
4.1.1	Bewertete Alternativen	26
4.1.2	Entscheidung	27
4.2	ORM: Entity Framework Core 9	27
4.2.1	Bewertete Alternativen	27
4.2.2	Entscheidung	28
4.3	Datenbank: PostgreSQL	28
4.3.1	Bewertete Alternativen	28
4.3.2	Entscheidung	28
4.4	iOS: Swift und SwiftUI	29
4.4.1	Bewertete Alternativen	29
4.4.2	Entscheidung	29
4.5	Nebenläufigkeit: Swift Concurrency (async/await)	29
4.5.1	Bewertete Alternativen	29
4.5.2	Entscheidung	30
5	Backend-Implementierung	31
5.1	Projektstruktur	31
5.2	Datenbankmodelle	31
5.2.1	Owner	31
5.2.2	Dog	31
5.2.3	Breed und LegalRestrictions	33
5.3	DoggoContext und Beziehungskonfiguration	33
5.4	EF Core Migrations	33
5.5	Breed-Seeding	35
5.6	Authentifizierung mit JWT	36
5.6.1	Passwort-Hashing	36
5.6.2	JWT-Ausstellung	36
5.7	Program.cs – Anwendungskonfiguration	36
5.8	Docker-Deployment	38
5.8.1	Dockerfile (Multi-Stage-Build)	38

5.8.2	Docker Compose	38
5.9	Controller-Implementierung	38
5.9.1	OwnersController	39
5.9.2	DogsController und Autorisierung	41
5.10	API-Dokumentation mit Swagger	42
6	iOS-Applikation	44
6.1	Architektur der iOS-App	44
6.2	Projektstruktur	44
6.3	MapKit-Integration	45
6.4	Standortdienste und Berechtigungen	46
6.5	Authentifizierung in der iOS-App	46
6.5.1	Login und Registrierung	46
6.5.2	Token-Persistenz im Keychain	47
6.6	Hundeprofilverwaltung	47
6.7	Anzeige gesetzlicher Vorschriften	49
6.8	HTTP-Kommunikation mit dem Backend	49
6.9	Fehlerbehandlung und Ladezustände	49
6.9.1	Fehlerdefinition	49
6.9.2	Ladezustand und Fehleranzeige im ViewModel	51
6.9.3	Alert-Anzeige in der View	51
7	Testing und Deployment	53
7.1	Teststrategie	53
7.2	Backend-Tests	53
7.2.1	Manuelle Tests mit Swagger und Postman	53
7.2.2	Unit-Tests mit xUnit	54
7.3	iOS-Tests	54
7.4	Integrationstests	54
7.5	Deployment-Verifikation	56
8	Zusammenfassung	57
8.1	Erreichte Ziele	57
8.2	Suboptimale Entscheidungen	57
8.3	Einschränkungen der Lösung	58
8.4	Erweiterungsmöglichkeiten	59
A	API-Endpunkte im Detail	60

Abbildungsverzeichnis

2.1	Use-Case-Diagramm – Backend und iOS-App	19
3.1	Gesamtarchitektur von DoggoGO	20
3.2	Interne Schichten des Backends	21
3.3	Entity-Relationship-Diagramm (DoggoGO-Datenbank)	22
3.4	Sequenzdiagramm – Authentifizierter API-Aufruf	23
5.1	Swagger-UI des DoggoGO-Backends	43
6.1	MVVM-Architektur der DoggoGO iOS-App	44

Tabellenverzeichnis

1.1	Vergleich bestehender Lösungen	12
2.1	Funktionale Anforderungen – Backend	16
2.2	Funktionale Anforderungen – iOS-App	17
2.3	Nicht-funktionale Anforderungen	18
3.1	Übersicht der API-Endpunkte	23
4.1	Vergleich Backend-Frameworks	27
4.2	Vergleich Datenbanksysteme	28
7.1	Getestete API-Szenarien	54

Listings

4.1	Swift Concurrency – <code>.task{}</code> in SwiftUI	30
5.1	Modell Owner.cs	32
5.2	Modell Dog.cs	32
5.3	Modelle Breed.cs und LegalRestrictions.cs	33
5.4	DoggoContext.cs – Beziehungskonfiguration (Auszug)	34
5.5	BreedSeeder.cs – Idempotentes Seeding	35
5.6	breeds.json – Auszug (Bullterrier)	35
5.7	Passwort-Hashing (Registrierung, Auszug)	36
5.8	JWT-Ausstellung (Login, Auszug)	36
5.9	Program.cs (vereinfacht)	37
5.10	Dockerfile – Multi-Stage-Build (Auszug)	38
5.11	compose.yaml	39
5.12	OwnersController – Register und Login (Auszug)	39
5.13	DogsController – OwnerId aus JWT-Claims (Auszug)	41
6.1	Kartenansicht in SwiftUI (vereinfacht)	45
6.2	LocationManager (Auszug)	46
6.3	Login-Anfrage (Auszug)	46
6.4	AuthService – Keychain-Speicherung (Auszug)	47
6.5	DogListView – Übersicht der Hundeprofile (Auszug)	48
6.6	LegalRestrictionsView – Anzeige der Vorschriften	49
6.7	APIService – Authentifizierter Request (Auszug)	50
6.8	APIError – Typisierte Fehlerdefinition	50
6.9	ViewModel – Ladezustand und Fehlerbehandlung	51
6.10	SwiftUI – Alert fuer Fehlermeldungen	51
7.1	xUnit-Test – Passwort-Hash-Verifikation	55
7.2	XCTest – AuthService Keychain-Test	56

Kapitel 1

Einleitung

1.1 Ausgangssituation und Problemstellung

Wer mit einem Hund in einer österreichischen Stadt oder Gemeinde unterwegs ist, sieht sich täglich mit einer Vielzahl praktischer Fragen konfrontiert: Wo befindet sich die nächste Freilaufwiese, auf der der Hund ohne Leine toben darf? Gibt es in der Nähe einen Hundekot-Müllbehälter oder einen Sackerlspender? Und – besonders relevant für Halterinnen und Halter bestimmter Rassen – gilt an diesem Ort eine Leinen- oder Maulkorbpflicht für ihren Hund?

Diese Informationen sind zwar grundsätzlich verfügbar, aber über viele verschiedene Quellen verstreut: kommunale Webseiten, amtliche Kundmachungen, geografische Open-Data-Portale oder mündliche Weitergabe unter Hundehalterinnen und Hundehaltern. Eine zentrale, standortbewusste und rassenspezifische Anlaufstelle fehlt im deutschsprachigen App-Markt weitgehend.

1.1.1 Defizite bestehender Lösungen

Eine Analyse verfügbarer Anwendungen zeigt, dass keine der bekannten Lösungen alle relevanten Anforderungen in einer einzigen App vereint:

Tabelle 1.1: Vergleich bestehender Lösungen

App / Dienst	Karte mit POIs	Rechtsvorgaben	Rassenfilter	iOS-App
BringGassi	Ja (wenige)	Nein	Nein	Ja
Google Maps	Nein	Nein	Nein	Ja
OpenStreetMap	Begrenzt	Nein	Nein	Indirekt
Wien.gv.at	Ja (Wien)	Begrenzt	Nein	Nein
DoggoGO	Ja	Ja	Ja	Ja

BringGassi bietet zwar eine kartenzentrierte Ansicht für Hundeausflüge, deckt jedoch keine rechtlichen Vorgaben ab und ist regional stark begrenzt. **Google Maps** erlaubt grundsätzlich die Suche nach Hundeauslaufzonen, verfügt aber über keinerlei rassenspezifische Rechtsinformationen und ist nicht auf Hundehaltung spezialisiert. Kommunale Portale wie das der Stadt Wien bieten Standortdaten für ihren Zuständigkeitsbereich, sind aber weder mobil optimiert noch rassenspezifisch und nicht überregional einsetzbar.

Das Kerndefizit aller betrachteten Lösungen liegt in der fehlenden Verknüpfung von drei Aspekten, die Hundehalterinnen und Hundehalter täglich benötigen:

1. standortbezogene Karte mit relevanten Points of Interest (POIs),
2. rassenspezifische gesetzliche Informationen (Leinen-/Maulkorbpflicht),
3. plattformübergreifende Verfügbarkeit (Web *und* iOS).

1.2 Zielsetzung dieser Arbeit

Diese Diplomarbeit behandelt die Realisierung von zwei zentralen Komponenten des DoggoGO-Projekts: dem **Backend** und der **iOS-Applikation**.

Das **Backend** hat folgende Aufgaben:

- Bereitstellung einer RESTful API für die Angular-Webanwendung und die iOS-App,
- Verwaltung von Hundehalterinnen und Hundehaltern sowie ihrer Hundepreise,
- Persistenz der Hunderassen inklusive ihrer rassenspezifischen Rechtsvorschriften,
- sicherer Betrieb in einer Docker-Container-Umgebung.

Die **iOS-App** soll:

- eine interaktive Karte mit standortbasierten Markierungen (Freilaufwiesen, Sackerlspender, Hundekot-Müllbehälter) anzeigen,
- das Hundeprofil der Nutzerin / des Nutzers verwalten,
- rassenspezifische Leinen- und Maulkorbvorschriften für den aktuellen Standort anzeigen,
- eine sichere Anmeldung per JWT-Authentifizierung bieten.

1.3 Abgrenzung

Die Angular-Webanwendung (inklusive UI/UX-Konzept, Routenplanung, Wetterintegration und Progressive-Web-App-Funktionalität) wurde von Manuel Rankl entwickelt und ist nicht Gegenstand dieser Arbeit. Gleiches gilt für die Entwicklung eines Android-Clients. Das in dieser Arbeit beschriebene Backend bildet die gemeinsame Datenbasis für beide Frontends.

1.4 Aufbau der Arbeit

Die vorliegende Arbeit ist wie folgt gegliedert:

Kapitel 2 – Anforderungsanalyse: Beschreibung der Stakeholder, funktionaler und nicht-funktionaler Anforderungen sowie der Use-Case-Szenarien aus Sicht des Backends und der iOS-App.

Kapitel 3 – Systemarchitektur: Gesamtarchitektur der Plattform, Datenbankdesign, REST-API-Überblick und Kommunikationsfluss zwischen den Systemkomponenten.

Kapitel 4 – Technologieauswahl: Begründete Auswahl der eingesetzten Technologien (ASP.NET Core 8, Entity Framework Core 9, PostgreSQL, iOS/SwiftUI) mit Bewertung von Alternativen.

Kapitel 5 – Backend-Implementierung: Detaillierte Beschreibung der API-Endpunkte, Datenbankmodelle, Migrations, Authentifizierung, Breed-Seeding und Docker-Deployment.

Kapitel 6 – iOS-Applikation: Architektur und Implementierung der iOS-App: MapKit-Integration, Standortdienste, Authentifizierungsflow und Hundeprofilverwaltung.

Kapitel 7 – Testing und Deployment: Teststrategie, durchgeführte Tests und Erkenntnisse aus dem Deployment.

Kapitel 8 – Zusammenfassung: Reflexion der erreichten Ziele, suboptimaler Entscheidungen, Einschränkungen und Erweiterungsmöglichkeiten.

Kapitel 2

Anforderungsanalyse

Dieses Kapitel beschreibt die Anforderungen an das Backend und die iOS-App von DoggoGO. Die Erhebung erfolgte auf Basis des Projektvorschlags, des Product Backlogs sowie eigener Überlegungen zu Sicherheit, Performance und Wartbarkeit.

2.1 Stakeholder und Zielgruppe

Die wichtigsten Stakeholder des Projekts sind:

Hundehalterinnen und Hundehalter (Endnutzerinnen/-nutzer) Personen, die mit einem oder mehreren Hunden in urbanen oder ländlichen Gebieten unterwegs sind und standortbezogene Informationen benötigen. Sie interagieren mit der Plattform über die Webapplikation oder die iOS-App.

Entwicklerinnen und Entwickler Das Projektteam (Manuel Rankl für das Angular-Frontend, Daniel Hoheneder für Backend und iOS). Anforderungen an Wartbarkeit, klare API-Contracts und gute Dokumentation sind daher ebenso relevant.

HTL Leonding (Auftraggeber) Die Schule als Auftraggeber und Bewerter der Diplomarbeit erwartet eine technisch fundierte, vollständig dokumentierte Lösung, die die im Projektvorschlag definierten Ziele erfüllt.

2.2 Funktionale Anforderungen

2.2.1 Backend-API

Tabelle 2.1: Funktionale Anforderungen – Backend

ID	Anforderung
FA-B01	Das System muss eine RESTful API bereitstellen, die von der Angular-Webanwendung und der iOS-App konsumiert werden kann.
FA-B02	Nutzerinnen und Nutzer müssen sich mit E-Mail-Adresse und Passwort registrieren und anmelden können.
FA-B03	Nach erfolgreicher Anmeldung muss ein JWT ausgestellt werden, das alle nachfolgenden Anfragen autorisiert.
FA-B04	Angemeldete Nutzerinnen und Nutzer müssen Hundeprofile (Name, Alter, Rasse) anlegen, lesen, aktualisieren und löschen können (CRUD).
FA-B05	Das System muss eine Liste aller verfügbaren Hunderassen mit deren Beschreibung und rassenspezifischen Rechtsvorschriften liefern.
FA-B06	Rasseninformationen müssen beim ersten Start aus einer JSON-Datei automatisch in die Datenbank eingelesen werden (Seeding).
FA-B07	Das Backend muss in einem Docker-Container lauffähig sein und mit einer ebenfalls containerisierten PostgreSQL-Instanz kommunizieren.
FA-B08	Die API-Endpunkte müssen über Swagger (OpenAPI) dokumentiert und testbar sein.

2.2.2 iOS-Applikation

Tabelle 2.2: Funktionale Anforderungen – iOS-App

ID	Anforderung
FA-I01	Die App muss eine interaktive Karte anzeigen (DOG1).
FA-I02	Der eigene Standort muss auf der Karte dargestellt werden (DOG2).
FA-I03	Nutzerinnen und Nutzer müssen sich in der App registrieren und anmelden können.
FA-I04	Angemeldete Nutzerinnen und Nutzer müssen Hundeprofile verwalten können.
FA-I05	Die App muss die Maulkorb- und Leinenpflicht für die gespeicherte Rasse und den aktuellen Standort anzeigen (DOG6).
FA-I06	Nutzerinnen und Nutzer müssen aus einer Liste von Hunderassen auswählen können (DOG8).
FA-I07	Die App muss mit dem Backend über die bereitgestellte REST-API kommunizieren.
FA-I08	Sitzungen müssen durch Speicherung des JWT persistent über App-Neustarts hinaus erhalten bleiben.

2.3 Nicht-funktionale Anforderungen

Tabelle 2.3: Nicht-funktionale Anforderungen

ID	Kategorie	Anforderung
NF-01	Sicherheit	Passwörter dürfen nicht im Klartext gespeichert werden. Es wird HMAC-SHA512 mit individuellem Salt eingesetzt.
NF-02	Sicherheit	JWTs müssen mit einem serverseitigen Secret signiert sein und eine konfigurierbare Ablaufzeit besitzen.
NF-03	Datenschutz	Die App darf Standortdaten nur mit expliziter Zustimmung der Nutzerin / des Nutzers erfassen (DSGVO, DOG12).
NF-04	Performance	API-Antworten müssen unter normalen Bedingungen in unter 500 ms eintreffen.
NF-05	Wartbarkeit	Der Quellcode muss in einem öffentlichen Git-Repository versioniert sein; Commits müssen nachvollziehbar beschrieben werden.
NF-06	Portierbarkeit	Das Backend muss plattformunabhängig über Docker deploybar sein.
NF-07	Kompatibilität	Die iOS-App muss auf Geräten ab iOS 16 lauffähig sein.
NF-08	Erweiterbarkeit	Die Datenbankmodelle und API-Endpunkte müssen so gestaltet sein, dass neue Entitäten (z. B. Standorte, Events) ohne Breaking Changes ergänzt werden können.

2.4 Use-Case-Szenarien

Abbildung 2.1 zeigt die wesentlichen Use Cases aus Sicht der Backend- und iOS-Komponenten.

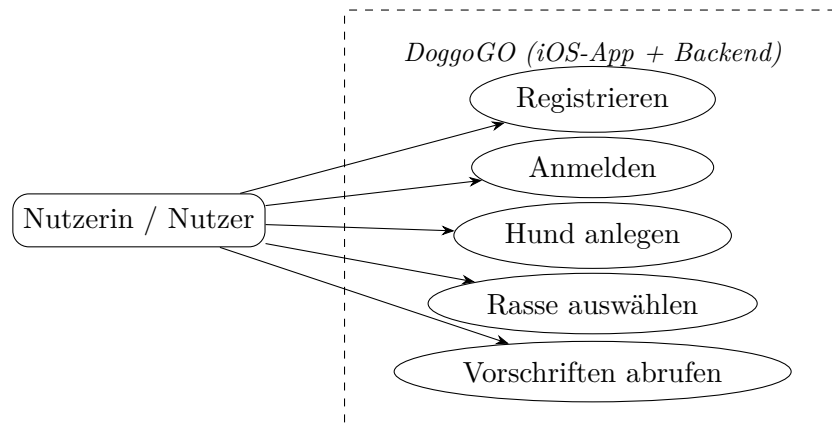


Abbildung 2.1: Use-Case-Diagramm – Backend und iOS-App

Die wichtigsten Szenarien sind:

UC-01: Registrierung Eine neue Nutzerin / ein neuer Nutzer öffnet die iOS-App, gibt Name, E-Mail und Passwort ein und sendet die Anfrage an `POST /api/owners/register`. Das Backend validiert die Eingaben, hashst das Passwort und legt einen neuen `Owner`-Datensatz an. Die App erhält eine Erfolgsantwort und leitet zur Anmeldeseite weiter.

UC-02: Anmeldung und Kartenzugriff Die Nutzerin / der Nutzer meldet sich mit E-Mail und Passwort an (`POST /api/owners/login`). Das Backend prüft den Passwort-Hash und stellt bei Übereinstimmung ein JWT aus. Die iOS-App speichert das Token im `Keychain` und nutzt es für alle weiteren Anfragen. Anschließend wird die Kartenansicht mit dem aktuellen Standort geladen.

UC-03: Hundeprofil anlegen Nach der Anmeldung wählt die Nutzerin / der Nutzer aus der Rassenliste eine Hunderasse aus (`GET /api/breeds`) und erstellt ein Hundeprofil (`POST /api/dogs`). Der Datensatz wird in der Datenbank gespeichert und der iOS-App bestätigt.

UC-04: Rechtliche Einschränkungen abrufen Die iOS-App ruft für die gespeicherte Hunderasse die `LegalRestrictions` ab und zeigt Leinen- und Maulkorbvorschriften für den aktuellen Standort an.

Kapitel 3

Systemarchitektur

3.1 Gesamtarchitektur

DoggoGO folgt einer klassischen **Client-Server-Architektur** mit drei Schichten: einem Web-Frontend (Angular), einer nativen iOS-App und einem gemeinsamen Backend-Server, der über eine REST-API kommuniziert. Die Datenhaltung erfolgt in einer PostgreSQL-Datenbank. Abbildung 3.1 illustriert die Systemkomponenten und deren Beziehungen.

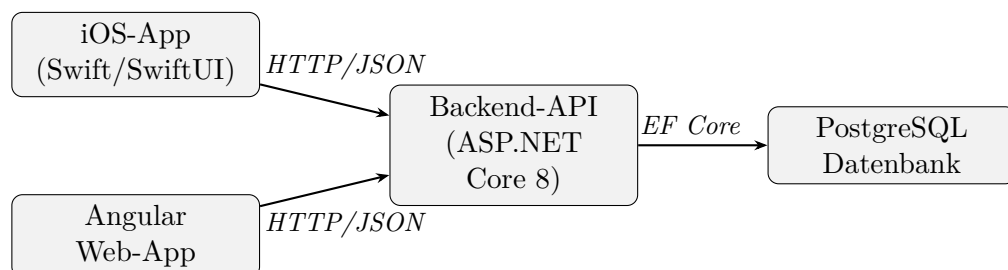


Abbildung 3.1: Gesamtarchitektur von DoggoGO

Die Entscheidung für eine gemeinsame Backend-API (anstelle von separaten Backends für Web und iOS) hat folgende Vorteile:

- Einmalige Implementierung der Geschäftslogik – keine Redundanz,
- einheitlicher Datenzugriff für beide Clients,
- einfache Erweiterbarkeit um weitere Clients (z. B. Android).

3.2 Schichten des Backends

Das Backend ist intern in drei Schichten gegliedert (Abbildung 3.2):

Controller-Schicht (Presentation Layer) ASP.NET Core Controller nehmen HTTP-Anfragen entgegen, validieren Eingaben und delegieren die Verarbeitung an die Service-/Logikschicht.

Modell- und Datenbankschicht (Data Layer) Entity Framework Core 9 übernimmt als ORM die Abbildung der C#-Objekte auf PostgreSQL-Tabellen. `DoggoContext` ist der zentrale `DbContext`.

Hilfsdienste (Utility) `BreedSeeder` importiert Hunderassen beim Start aus `breeds.json` in die Datenbank, sofern noch keine Einträge vorhanden sind.

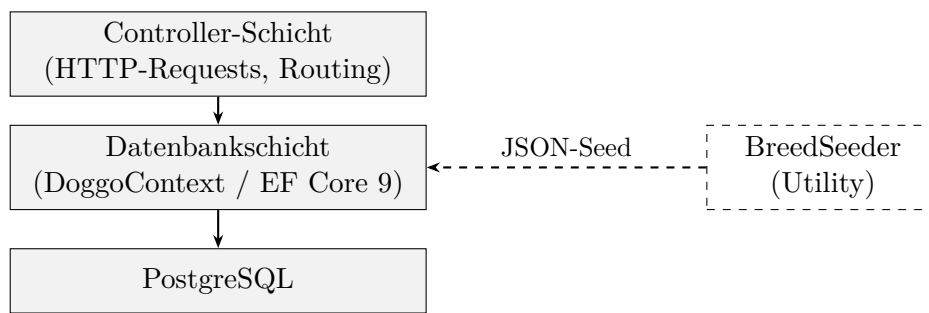


Abbildung 3.2: Interne Schichten des Backends

3.3 Datenbankdesign

Das Datenbankschema besteht aus vier Entitäten (Abbildung 3.3):

Owner Repräsentiert eine registrierte Nutzerin / einen registrierten Nutzer mit Anmeldedaten (E-Mail, Passwort-Hash, Passwort-Salt) und optionaler Telefonnummer.

Dog Ein Hundeprofil mit Name und Alter, das einer Nutzerin / einem Nutzer (`OwnerId`) und einer Hunderasse (`BreedId`) zugeordnet ist.

Breed Eine Hunderasse mit Bezeichnung und Beschreibung, verknüpft mit einem `LegalRestrictions`-Eintrag.

LegalRestrictions Rechtskategorie (z. B. `spezielle_hunderasse`) und Liste konkreter Vorschriften (Leinen-/Maulkorbpflicht, ATP-Pflicht etc.) als PostgreSQL-Array.

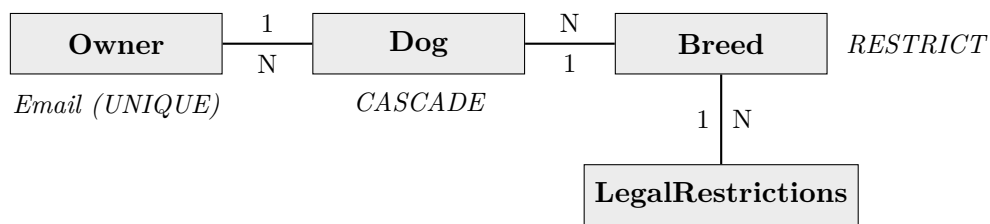


Abbildung 3.3: Entity-Relationship-Diagramm (DoggoGO-Datenbank)

3.3.1 Beziehungen und Löscherhalten

Die Beziehungen zwischen den Entitäten wurden im `DoggoContext` mit `OnModelCreating` explizit konfiguriert:

- **Owner → Dog (1:N, Cascade Delete):** Wird eine Nutzerin / ein Nutzer gelöscht, werden alle zugehörigen Hundeprofile ebenfalls gelöscht. Dies entspricht dem erwarteten Verhalten beim Account-Delete.
- **Dog → Breed (N:1, Restrict Delete):** Eine Hunderasse kann nicht gelöscht werden, solange noch Hunde dieser Rasse existieren. Damit wird die referenzielle Integrität gewahrt.
- **Breed → LegalRestrictions (N:1, Restrict Delete):** Analog – ein Rechtsdatensatz kann nicht entfernt werden, solange Rassen darauf verweisen.
- **Owner.Email (Unique Index):** Verhindert mehrfache Registrierung mit derselben E-Mail-Adresse auf Datenbankebene.

3.4 REST-API-Überblick

Die API folgt den REST-Prinzipien: Ressourcen werden über hierarchische URLs adressiert, HTTP-Methoden spiegeln die CRUD-Operationen wider, und alle Antworten erfolgen im JSON-Format. Tabelle 3.1 listet die geplanten Endpunkte auf.

Tabelle 3.1: Übersicht der API-Endpunkte

Methode	Pfad	Beschreibung
POST	/api/owners/register	Neue Nutzerin / neuen Nutzer registrieren
POST	/api/owners/login	Anmelden, JWT erhalten
GET	/api/owners/{id}	Nutzerprofil abrufen (auth. erforderlich)
GET	/api/dogs	Alle eigenen Hunde abrufen
POST	/api/dogs	Neuen Hund anlegen
PUT	/api/dogs/{id}	Hundeprofi aktualisieren
DELETE	/api/dogs/{id}	Hundeprofi löschen
GET	/api/breeds	Alle Hunderassen abrufen
GET	/api/breeds/{id}	Einzelne Rasse mit Rechtsvorschriften abrufen

3.5 Kommunikationsfluss

Abbildung 3.4 zeigt den typischen Ablauf einer authentifizierten Anfrage von der iOS-App an das Backend:

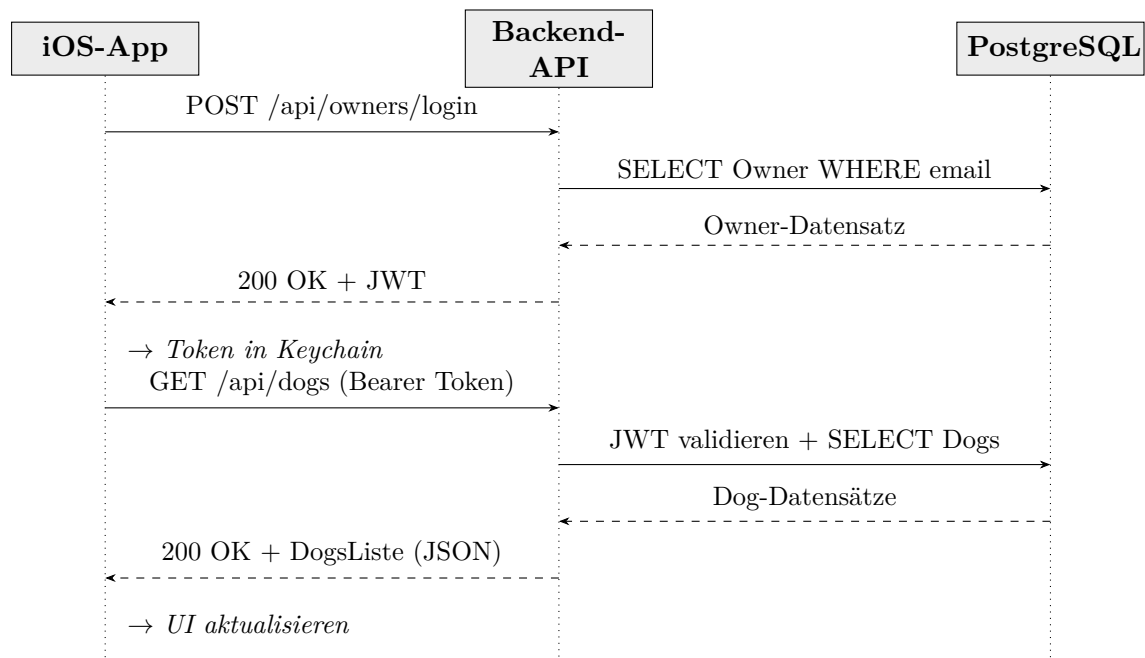


Abbildung 3.4: Sequenzdiagramm – Authentifizierter API-Aufruf

1. Die iOS-App sendet `POST /api/owners/login` mit Credentials im Request-Body.

2. Das Backend prüft den HMAC-SHA512-Hash und gibt bei Erfolg ein signiertes JWT zurück.
3. Die App speichert das Token sicher im iOS Keychain.
4. Für alle weiteren Anfragen wird das Token im **Authorization-Header** (**Bearer <token>**) mitgesendet.
5. Der ASP.NET Core Middleware-Stack validiert das Token vor jedem geschützten Endpunkt.

3.6 Sicherheitsarchitektur

Sicherheit ist ein Querschnittsthema, das alle Schichten des Systems betrifft. Für DoggoGO wurden Maßnahmen auf drei Ebenen getroffen: Passwortspeicherung, Tokenbasierte Authentifizierung und Autorisierung auf Datenschutzebene.

Passwortspeicherung Passwörter werden niemals im Klartext gespeichert. Stattdessen wird für jede Nutzerin / jeden Nutzer beim Registrieren ein individueller zufälliger Salt erzeugt (**HMACSHA512.Key**), und das Passwort wird mit diesem Salt gehasht. Da jeder Salt einzigartig ist, erzeugt dasselbe Passwort für unterschiedliche Nutzerinnen und Nutzer stets unterschiedliche Hashwerte. Damit sind vorberechnete Regenbogentabellen (*Rainbow Tables*) wirkungslos. Salt und Hash werden als **byte[]**-Felder in der Datenbank persistiert.

JWT-Tokenfluss Nach erfolgreicher Anmeldung stellt der Server ein **JSON Web Token** (JWT) aus. Das Token ist mit einem serverseitigen Secret (HMAC-SHA256) signiert und trägt als Payload die **OwnerId** sowie die E-Mail-Adresse. Es hat eine konfigurierbare Ablaufzeit (Standard: 8 Stunden). Der Client sendet das Token bei jeder nachfolgenden Anfrage im **Authorization-Header** (**Bearer <token>**). Der ASP.NET Core Middleware-Stack validiert die Signatur und den Ablaufzeitstempel, bevor die Controller-Methode aufgerufen wird – manipulierte oder abgelaufene Tokens werden mit **401 Unauthorized** abgelehnt.

Autorisierung auf Datenschutzebene Die **OwnerId** wird serverseitig aus den verifizierten JWT-Claims gelesen, nie aus dem Request-Body. Dadurch ist es technisch unmöglich, dass eine angemeldete Nutzerin / ein angemeldeter Nutzer Daten einer anderen Person liest oder verändert: Jede Datenbankabfrage filtert implizit nach der eigenen **OwnerId**.

CORS Die API ist so konfiguriert, dass nur definierte Origins (Angular-Frontend und iOS-App) Cross-Origin-Anfragen stellen dürfen. Damit werden unberechtigte Zugriffe von fremden Webseiten auf die API unterbunden.

3.7 Deployment-Architektur

Das gesamte Backend-Deployment basiert auf **Docker Compose**. Die `compose.yaml`-Datei definiert zwei Services: `postgres` (offizielle PostgreSQL-Image) und `backend` (das ASP.NET Core-Anwendungs-Image). Beide Services kommunizieren über ein gemeinsames Docker-Netzwerk; das Backend verwendet die Umgebungsvariablen des Compose-Stacks für die Datenbankverbindung. Die PostgreSQL-Daten werden in einem benannten Volume (`postgres_data`) persistiert.

Kapitel 4

Technologieauswahl

Die Wahl geeigneter Technologien hat einen entscheidenden Einfluss auf Wartbarkeit, Performance und Entwicklungsgeschwindigkeit eines Projekts. Dieses Kapitel begründet die getroffenen Entscheidungen für Backend und iOS-App und bewertet jeweils relevante Alternativen.

4.1 Backend-Framework: ASP.NET Core 8

4.1.1 Bewertete Alternativen

Für die Implementierung der REST-API wurden drei Backend-Frameworks in Betracht gezogen:

Tabelle 4.1: Vergleich Backend-Frameworks

Kriterium	ASP.NET Core 8	Node.js / Express	Spring Boot 3
Sprache	C# (.NET)	JavaScript / TypeScript	Java / Kotlin
Performance	Sehr hoch (Benchmark: Top-3 TechEmpower)	Hoch (Event Loop)	Hoch
ORM-Unterstützung	Entity Framework Core (erstklassig)	Prisma / TypeORM	Hibernate / Spring Data
Lernkurve	Mittel (C# Vorwissen vorhanden)	Niedrig (JS bekannt)	Hoch
Docker-Support	Offizielles .NET-Image, klein	Node-Image	JVM-Image, groß
Schul-Kontext	HTL-Lehrplan (C#)	Partiell	Nein

4.1.2 Entscheidung

ASP.NET Core 8 wurde gewählt, weil C# im HTL-Lehrplan fest verankert ist und bereits umfangreiche Vorkenntnisse vorhanden sind. Das Framework bietet erstklassige Integration mit Entity Framework Core, sehr gute Performance und einen klar strukturierten Middleware-Stack. Die offiziellen Microsoft-Docker-Images sind schlank und werden langfristig gewartet. Spring Boot wäre eine technisch gleichwertige Alternative, erfordert jedoch Java und eine deutlich steilere Einarbeitungskurve in das Spring-Ökosystem. Node.js / Express wurde verworfen, da der Typsicherheitsgewinn gegenüber JavaScript und die bessere ORM-Integration von EF Core für ein datenbankzentrisches Projekt ausschlaggebend war.

4.2 ORM: Entity Framework Core 9

4.2.1 Bewertete Alternativen

Für den Datenbankzugriff wurden drei Ansätze verglichen:

Entity Framework Core 9 (EF Core) Vollständiges ORM mit Code-First-Migrations, LINQ-Abfrageunterstützung und Change Tracking.

Dapper Lightweight Micro-ORM: Nur SQL-Mapping, kein Change Tracking, keine automatischen Migrations.

ADO.NET (direkt) Rohe Datenbankzugriffe; maximale Kontrolle, aber hoher Boilerplate-Aufwand.

4.2.2 Entscheidung

EF Core 9 wurde bevorzugt, weil es automatische Migrationen ermöglicht (kein manuelles SQL für Schemaänderungen), LINQ-Abfragen typsicher und intellisensefähig macht und sehr gut mit ASP.NET Core zusammenspielt. Für einfache CRUD-Operationen wie in DoggoGO ist der Overhead von EF Core vernachlässigbar. Dapper wäre bei komplexen, performance-kritischen Datenbankabfragen sinnvoller; da DoggoGO aktuell keine komplexen Joins oder Massenoperationen benötigt, ist dieser Vorteil irrelevant.

4.3 Datenbank: PostgreSQL

4.3.1 Bewertete Alternativen

Tabelle 4.2: Vergleich Datenbanksysteme

Kriterium	PostgreSQL	MySQL / MariaDB	SQLite
ACID-Konformität	Vollständig	Vollständig	Vollständig
Array-Typ (<code>text[]</code>)	Nativ unterstützt	Nicht nativ	Nicht vorhanden
Docker-Image	Offiziell, stabil	Offiziell	Nicht nötig
Skalierbarkeit	Hoch	Hoch	Niedrig (Datei-basiert)
EF Core Support	Npgsql (erstklassig)	Pomelo (gut)	Eingebaut

4.3.2 Entscheidung

PostgreSQL wurde gewählt, weil das Modell `LegalRestrictions` die Regeln als `List<string>` speichert – ein Feature, das in PostgreSQL nativ als `text[]` (Array-Typ) unterstützt wird und von Npgsql direkt auf C#-Listen gemappt werden kann. MySQL / MariaDB bietet diesen nativen Array-Typ nicht; dort wäre eine separate Normalisierungstabelle oder JSON-Serialisierung notwendig gewesen, was den Code komplexer gemacht hätte. SQLite schied aus, da die Datei-basierte Architektur für eine containerisierte Multi-User-Anwendung nicht geeignet ist.

4.4 iOS: Swift und SwiftUI

4.4.1 Bewertete Alternativen

Für die native iOS-App wurden drei Ansätze verglichen:

Swift / SwiftUI (nativ) Offizieller Apple-Stack, maximale Plattformintegration, beste MapKit- und Keychain-Unterstützung.

React Native JavaScript-basiert, Cross-Platform (iOS + Android). MapKit-Unterstützung über Drittbibliotheken, weniger tief integriert.

Flutter Dart-basiert, Cross-Platform. Google-Maps-Integration gut; Apple-Ökosystem (Keychain, HealthKit etc.) schwieriger anzubinden.

4.4.2 Entscheidung

Da das Projektziel explizit eine **iOS**-Applikation vorsieht (kein Android-Support im Projektrahmen) und MapKit sowie der iOS **Keychain** für Kartendarstellung und sichere Token-Speicherung zentrale Anforderungen sind, fiel die Wahl auf den nativen **Swift / SwiftUI**-Stack. SwiftUI ermöglicht eine deklarative UI-Entwicklung mit weniger Boilerplate als UIKit und integriert sich nahtlos in das Apple-Ökosystem. Cross-Platform-Frameworks wie React Native oder Flutter bieten für reine iOS-Anforderungen keinen Mehrwert, führen aber zu einer tieferen Abstraktionsschicht und eingeschränkterem Zugang zu nativen APIs.

4.5 Nebenläufigkeit: Swift Concurrency (async/await)

4.5.1 Bewertete Alternativen

Netzwerkanfragen in iOS-Apps dürfen den Hauptthread (UI-Thread) nicht blockieren, da sonst die Benutzeroberfläche einfriert. Historisch standen drei Ansätze zur Verfügung:

Completion Handler (Callbacks) Die klassische Lösung: `URLSession.dataTask(with:completionHandler:)` nimmt eine Closure entgegen, die nach Abschluss der Anfrage aufgerufen wird. Bei verketteten Anfragen entsteht schnell *Callback Hell* – tief verschachtelte Closures, die schwer lesbar und fehleranfällig sind.

Combine Framework Apples reaktives Framework ermöglicht deklarative Datenpipelines mit **Publisher** und **Subscriber**. Combine ist mächtig, hat aber eine steile Lernkurve und erzeugt bei einfachen HTTP-Anfragen viel Boilerplate

(AnyCancellable, sink, receive(on:)).

Swift Concurrency (async/await) Seit Swift 5.5 (iOS 15+) bietet die Sprache natives `async/await`. Damit lassen sich asynchrone Operationen sequenziell und lesbar formulieren, ohne Callback-Verschachtelung. `URLSession` unterstützt `async/await` nativ seit iOS 15.

4.5.2 Entscheidung

Swift Concurrency wurde gewählt, weil die Zielplattform iOS 16+ ist (Anforderung NF-07) und daher `async/await` in vollem Umfang verfügbar ist. Der Code ist gegenüber Completion-Handlern deutlich lesbarer – ein `try await URLSession.shared.data(for` ersetzt eine dreistufige Callback-Kette vollständig. Gegenüber Combine entfällt der Aufwand für das Subscription-Management; für die DoggoGO-Anwendungsfälle (einfache Request-Response-Zyklen ohne komplexe Datenpipelines) ist Combine überdimensioniert. Zusätzlich erlaubt das `.task{}`-Modifer in SwiftUI das automatische Starten und Abbrechen von asynchronen Aufgaben in Abhängigkeit vom View-Lebenszyklus, was die Integration mit dem MVVM-Pattern vereinfacht:

```
1 .task {  
2     await viewModel.loadDogs() // startet beim Erscheinen, bricht  
3 }                               // ab wenn die View verschwindet
```

Listing 4.1: Swift Concurrency – `.task{}` in SwiftUI

Kapitel 5

Backend-Implementierung

5.1 Projektstruktur

Das Backend-Projekt (`Backend.csproj`) ist als ASP.NET Core 8 Web-API angelegt und folgt einer klaren Verzeichnisstruktur:

- `Models/` – C#-Entitätsklassen (`Dog`, `Owner`, `Breed`, `LegalRestrictions`)
- `Data/` – `DoggoContext` (`DbContext`) sowie Seed-Daten als JSON
- `Migrations/` – automatisch generierte EF Core-Migrationsdateien
- `Utility/` – Hilfsdienste (`BreedSeeder`)
- `Program.cs` – Anwendungseinstiegspunkt und Service-Konfiguration
- `Dockerfile` – Multi-Stage-Build für Docker-Deployment

5.2 Datenbankmodelle

5.2.1 Owner

`Owner` repräsentiert eine registrierte Nutzerin / einen registrierten Nutzer. Passwörter werden niemals im Klartext gespeichert; stattdessen werden `PasswordHash` und `PasswordSalt` als `byte[]` persistiert.

5.2.2 Dog

`Dog` hält `Name`, `Alter` sowie Fremdschlüssel zu `Owner` und `Breed`. Navigations-Properties ermöglichen EF Core, die Daten bei Bedarf per `Include()` einzulesen.


```
1 public class Owner
2 {
3     public Guid Id { get; set; }
4     public string Name { get; set; }
5     public string Email { get; set; }
6     public byte[] PasswordHash { get; set; }
7     public byte[] PasswordSalt { get; set; }
8     public string? PhoneNumber { get; set; } = null!;
9
10    // Navigation property
11    public ICollection<Dog> Dogs { get; set; } = new List<Dog>();
12 }
```

Listing 5.1: Modell Owner.cs

```
1 public class Dog
2 {
3     public Guid Id { get; set; }
4     public string Name { get; set; }
5     public int Age { get; set; }
6     public Guid BreedId { get; set; }
7     public Guid OwnerId { get; set; }
8
9     public Owner Owner { get; set; } = null!;
10    public Breed Breed { get; set; } = null!;
11 }
```

Listing 5.2: Modell Dog.cs

5.2.3 Breed und LegalRestrictions

Breed verknüpft einen Rassennamen mit einer Beschreibung und einem LegalRestrictions-Objekt. Letzteres speichert die Vorschriften als `List<string>`, die Npgsql auf `text []` in PostgreSQL mappt.

```
1 public class Breed
2 {
3     public Guid Id { get; set; }
4     public string BreedName { get; set; } = null!;
5     public string Description { get; set; } = null!;
6     public Guid LegalRestrictionsId { get; set; }
7     public LegalRestrictions LegalRestrictions { get; set; } = null
8     !;
9 }
10 public class LegalRestrictions
11 {
12     public Guid Id { get; set; }
13     public string Category { get; set; } = null!;
14     public List<string> Rules { get; set; } = new();
15 }
```

Listing 5.3: Modelle Breed.cs und LegalRestrictions.cs

5.3 DoggoContext und Beziehungskonfiguration

Der `DoggoContext` erbt von `DbContext` und registriert alle vier `DbSet`-Eigenschaften. Die Beziehungen und Constraints werden in `OnModelCreating` explizit konfiguriert, um keine unerwünschten Standardverhalten zu riskieren:

5.4 EF Core Migrations

Das Datenbankschema wird über EF Core Code-First-Migrations verwaltet. Die Initialmigration `InitDatabase` erstellt die vier Tabellen `LegalRestrictions`, `Owners`, `Breeds` und `Dogs` mit allen Constraints und Foreign-Keys. Neue Schemaänderungen (z. B. neue Spalten oder Entitäten) werden durch `dotnet ef migrations add <Name>` erzeugt und durch `dotnet ef database update` auf die laufende Datenbank angewendet. Beim Start der Anwendung wird die Datenbank automatisch migriert.

```
1 protected override void OnModelCreating(ModelBuilder modelBuilder)
2 {
3     // Owner -> Dog: 1:N, Cascade Delete
4     modelBuilder.Entity<Dog>()
5         .HasOne(d => d.Owner)
6         .WithMany(o => o.Dogs)
7         .HasForeignKey(d => d.OwnerId)
8         .OnDelete(DeleteBehavior.Cascade);
9
10    // Dog -> Breed: N:1, Restrict Delete
11    modelBuilder.Entity<Dog>()
12        .HasOne(d => d.Breed)
13        .WithMany()
14        .HasForeignKey(d => d.BreedId)
15        .OnDelete(DeleteBehavior.Restrict);
16
17    // Unique Index auf Owner.Email
18    modelBuilder.Entity<Owner>()
19        .HasIndex(o => o.Email)
20        .IsUnique();
21
22    // Breed -> LegalRestrictions: N:1, Restrict Delete
23    modelBuilder.Entity<Breed>()
24        .HasOne(b => b.LegalRestrictions)
25        .WithMany()
26        .HasForeignKey(b => b.LegalRestrictionsId)
27        .OnDelete(DeleteBehavior.Restrict);
28 }
```

Listing 5.4: DoggoContext.cs – Beziehungskonfiguration (Auszug)

5.5 Breed-Seeding

Damit beim ersten Start sofort eine vollständige Rassenliste verfügbar ist, liest der `BreedSeeder` beim Anwendungsstart `Data/breeds.json` ein und befüllt die Tabellen `LegalRestrictions` und `Breeds`. Der Seeder prüft zunächst, ob bereits Einträge vorhanden sind, um eine Mehrfach-Einspielung zu verhindern:

```
1 public static async Task SeedBreedsAsync(  
2     DoggoContext context, string jsonFilePath)  
3 {  
4     if (context.Breeds.Any()) return; // bereits befüllt  
5  
6     var jsonData = await File.ReadAllTextAsync(jsonFilePath);  
7     var breeds = JsonSerializer.Deserialize<List<Breed>>(jsonData,  
8         new JsonSerializerOptions { PropertyNameCaseInsensitive =  
9         true });  
10  
11     if (breeds != null)  
12     {  
13         await context.Breeds.AddRangeAsync(breeds);  
14         await context.SaveChangesAsync();  
15     }
```

Listing 5.5: BreedSeeder.cs – Idempotentes Seeding

Die JSON-Datei enthält pro Rasse einen `breedName`, eine `description` sowie ein `legal_restrictions`-Objekt mit `category` und einem `rules`-Array. Listing 5.6 zeigt einen Auszug:

```
1 {  
2     "breedName": "Bullterrier",  
3     "description": "Muskuloeser Hund mit eifoermigem Kopf.",  
4     "legal_restrictions": {  
5         "category": "spezielle_hunderasse",  
6         "rules": [  
7             "Mindestalter des Halters: 16 Jahre",  
8             "Sachkundeausbildung erforderlich",  
9             "Alltagstauglichkeitspruefung (ATP) verpflichtend",  
10            "Leinen- und Maulkorbpflicht im oeffentlichen Raum"  
11        ]  
12    }  
13 }
```

Listing 5.6: breeds.json – Auszug (Bullterrier)

5.6 Authentifizierung mit JWT

Die Authentifizierung basiert auf **JSON Web Tokens (JWT)**. Der Ablauf gliedert sich in zwei Schritte: Registrierung und Anmeldung.

5.6.1 Passwort-Hashing

Beim Registrieren wird das Passwort mit **HMAC-SHA512** und einem zufällig generierten Salt gehasht. Sowohl Hash als auch Salt werden als `byte[]` in der **Owner**-Entität gespeichert. Das Klartext-Passwort verlässt den Request-Scope nie:

```
1 using var hmac = new HMACSHA512();
2 owner.PasswordSalt = hmac.Key;
3 owner.PasswordHash = hmac.ComputeHash(
4     Encoding.UTF8.GetBytes(registerDto.Password));
```

Listing 5.7: Passwort-Hashing (Registrierung, Auszug)

5.6.2 JWT-Ausstellung

Nach erfolgreicher Anmeldung wird ein JWT mit konfigurierbarer Ablaufzeit und dem serverseitigen Secret signiert. Das Token enthält den **NameIdentifier**-Claim mit der Owner-ID:

```
1 var claims = new[]
2 {
3     new Claim(ClaimTypes.NameIdentifier, owner.Id.ToString()),
4     new Claim(ClaimTypes.Email, owner.Email)
5 };
6 var key = new SymmetricSecurityKey(
7     Encoding.UTF8.GetBytes(configuration["Jwt:Secret"]!));
8 var token = new JwtSecurityToken(
9     issuer: configuration["Jwt:Issuer"],
10    audience: configuration["Jwt:Audience"],
11    claims: claims,
12    expires: DateTime.UtcNow.AddHours(8),
13    signingCredentials: new SigningCredentials(
14        key, SecurityAlgorithms.HmacSha256));
```

Listing 5.8: JWT-Ausstellung (Login, Auszug)

5.7 Program.cs – Anwendungskonfiguration

`Program.cs` konfiguriert die Service-Registrierungen und die Middleware-Pipeline:

```
1 var builder = WebApplication.CreateBuilder(args);
2
3 builder.Services.AddDbContext<DoggoContext>(options =>
4     options.UseNpgsql(
5         builder.Configuration.GetConnectionString("
6             DefaultConnection"))));
7
8 builder.Services.AddControllers();
9 builder.Services.AddEndpointsApiExplorer();
10 builder.Services.AddSwaggerGen();
11
12 var app = builder.Build();
13
14 // Datenbankmigrationen und Seeding beim Start
15 using (var scope = app.Services.CreateScope())
16 {
17     var context = scope.ServiceProvider
18         .GetRequiredService<DoggoContext>();
19     context.Database.Migrate();
20     await BreedSeeder.SeedBreedsAsync(context, "Data/breeds.json");
21 }
22
23 app.UseSwagger();
24 app.UseSwaggerUI();
25 app.UseHttpsRedirection();
26 app.UseAuthentication();
27 app.UseAuthorization();
28 app.MapControllers();
29 app.Run();
```

Listing 5.9: Program.cs (vereinfacht)

5.8 Docker-Deployment

5.8.1 Dockerfile (Multi-Stage-Build)

Das `Dockerfile` verwendet einen Multi-Stage-Build, um ein möglichst kleines Produktions-Image zu erzeugen. In der Build-Stage wird das Projekt mit dem .NET 8 SDK kompiliert; in der finalen Stage wird nur die ASP.NET Runtime benötigt:

```
1 FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS base
2 WORKDIR /app
3 EXPOSE 8080
4
5 FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
6 WORKDIR /src
7 COPY ["Backend/Backend.csproj", "Backend/"]
8 RUN dotnet restore "Backend/Backend.csproj"
9 COPY . .
10 WORKDIR "/src/Backend"
11 RUN dotnet build "./Backend.csproj" -c Release -o /app/build
12
13 FROM build AS publish
14 RUN dotnet publish "./Backend.csproj" -c Release -o /app/publish
15
16 FROM base AS final
17 WORKDIR /app
18 COPY --from=publish /app/publish .
19 ENTRYPOINT ["dotnet", "Backend.dll"]
```

Listing 5.10: Dockerfile – Multi-Stage-Build (Auszug)

5.8.2 Docker Compose

`compose.yaml` startet zwei Services: die PostgreSQL-Datenbank und das Backend. Das Datenbank-Volume `postgres_data` sorgt dafür, dass Daten zwischen Container-Neustarts erhalten bleiben:

Mit `docker compose up -build` werden beide Services gestartet. Das Backend wartet durch den `depends_on`-Eintrag auf die Datenbank, bevor die Verbindung aufgebaut wird.

5.9 Controller-Implementierung

Die eigentliche HTTP-Schnittstelle des Backends wird durch ASP.NET Core Controller realisiert. Jeder Controller erbt von `ControllerBase` und wird mit den Attributen `[ApiController]` und `[Route(„api/[controller]“)]` annotiert. `[ApiController]`

```
1 version: '3.8'
2 services:
3   postgres:
4     image: postgres:latest
5     environment:
6       POSTGRES_USER: doggo
7       POSTGRES_PASSWORD: doggoG0123
8       POSTGRES_DB: doggo_db
9     ports:
10      - "5432:5432"
11     volumes:
12      - postgres_data:/var/lib/postgresql/data
13   backend:
14     build: .
15     ports:
16      - "8080:8080"
17     depends_on:
18      - postgres
19     environment:
20       ConnectionStrings__DefaultConnection: >
21         Host=postgres;Port=5432;Database=doggo_db;
22         Username=doggo;Password=doggoG0123
23
24 volumes:
25   postgres_data:
```

Listing 5.11: compose.yaml

aktiviert automatische Modellvalidierung und standardisierte Fehlerresponses; [Route] legt das URL-Präfix fest (z. B. /api/owners, /api/dogs).

5.9.1 OwnersController

Der OwnersController stellt die Endpunkte für Registrierung und Anmeldung bereit. Beide Aktionen sind öffentlich zugänglich (kein [Authorize]-Attribut), da eine Authentifizierung vor der Anmeldung nicht möglich ist:

```
1 [ApiController]
2 [Route("api/[controller]")]
3 public class OwnersController : ControllerBase
4 {
5     private readonly DoggoContext _context;
6     private readonly IConfiguration _config;
7
8     public OwnersController(DoggoContext context, IConfiguration
9     config)
10    {
11        _context = context;
12        _config = config;
```



```
12     }
13
14     [HttpPost("register")]
15     public async Task<IActionResult> Register(RegisterDto dto)
16     {
17         if (await _context.Owners.AnyAsync(o => o.Email == dto.
18             Email))
19             return Conflict("E-Mail-Adresse bereits vergeben.");
20
21         using var hmac = new HMACSHA512();
22         var owner = new Owner
23         {
24             Id = Guid.NewGuid(),
25             Name = dto.Name,
26             Email = dto.Email,
27             PasswordSalt = hmac.Key,
28             PasswordHash = hmac.ComputeHash(
29                 Encoding.UTF8.GetBytes(dto.Password)
30             );
31         };
32         _context.Owners.Add(owner);
33         await _context.SaveChangesAsync();
34         return CreatedAtAction(nameof(GetById),
35             new { id = owner.Id }, owner);
36     }
37
38     [HttpPost("login")]
39     public async Task<IActionResult> Login(LoginDto dto)
40     {
41         var owner = await _context.Owners
42             .FirstOrDefaultAsync(o => o.Email == dto.Email);
43         if (owner == null) return Unauthorized();
44
45         using var hmac = new HMACSHA512(owner.PasswordSalt);
46         var hash = hmac.ComputeHash(
47             Encoding.UTF8.GetBytes(dto.Password));
48         if (!hash.SequenceEqual(owner.PasswordHash))
49             return Unauthorized();
50
51         return Ok(new { token = CreateJwt(owner) });
52     }
53 }
```

Listing 5.12: OwnersController – Register und Login (Auszug)

Die Methode `Register` prüft zunächst per LINQ-Query, ob die E-Mail-Adresse bereits vergeben ist, und antwortet mit `409 Conflict`, falls dies der Fall ist. Bei der Anmeldung (`Login`) wird das eingegebene Passwort mit dem gespeicherten HMAC-SHA512-Hash verglichen. Stimmen sie nicht überein, liefert der Endpunkt `401 Unauthorized` ohne weitere Details – eine bewusste Designentscheidung, um keine Hinweise darauf zu geben, ob die E-Mail-Adresse registriert ist.

5.9.2 DogsController und Autorisierung

Alle Endpunkte des `DogsController` sind mit `[Authorize]` geschützt. Das bedeutet, dass der ASP.NET Core Middleware-Stack das `Authorization-Header-Token` vor jedem Aufruf validiert. Ist kein gültiges JWT vorhanden, antwortet die API automatisch mit `401 Unauthorized`, bevor die Controllermethode überhaupt ausgeführt wird.

Um sicherzustellen, dass Nutzerinnen und Nutzer ausschließlich ihre eigenen Hundepprofile verwalten können, wird die `OwnerId` nicht aus dem Request-Body gelesen, sondern direkt aus den JWT-Claims extrahiert:

```
1 [ApiController]
2 [Route("api/[controller]")]
3 [Authorize]
4 public class DogsController : ControllerBase
5 {
6     private readonly DoggoContext _context;
7
8     public DogsController(DoggoContext context)
9         => _context = context;
10
11     private Guid GetCurrentOwnerId() =>
12         Guid.Parse(User.FindFirstValue(ClaimTypes.NameIdentifier)!);
13
14     [HttpGet]
15     public async Task<IActionResult> GetAll()
16     {
17         var ownerId = GetCurrentOwnerId();
18         var dogs = await _context.Dogs
19             .Where(d => d.OwnerId == ownerId)
20             .Include(d => d.Breed)
21             .ToListAsync();
22         return Ok(dogs);
23     }
24 }
```

```
25 [HttpDelete("{id}")]
26 public async Task<IActionResult> Delete(Guid id)
27 {
28     var ownerId = GetCurrentOwnerId();
29     var dog = await _context.Dogs
30         .FirstOrDefaultAsync(d => d.Id == id
31                                 && d.OwnerId == ownerId);
32     if (dog == null) return NotFound();
33
34     _context.Dogs.Remove(dog);
35     await _context.SaveChangesAsync();
36     return NoContent();
37 }
38 }
```

Listing 5.13: DogsController – OwnerId aus JWT-Claims (Auszug)

Die Hilfsmethode `GetCurrentOwnerId()` liest den `NameIdentifier`-Claim aus dem validierten JWT. Da dieser Claim beim Login serverseitig eingetragen wird (Listing 5.8), ist er fälschungssicher – der Client kann die `OwnerId` nicht manipulieren. Das `Delete`-Beispiel zeigt außerdem, wie eine Berechtigungsprüfung auf Datensatzebene erfolgt: Die LINQ-Query filtert gleichzeitig nach `Id` und `OwnerId`, sodass ein fremdes Hundeprofil zuverlässig `404 NotFound` zurückgibt, statt eine Fehlermeldung über fehlende Rechte preiszugeben.

5.10 API-Dokumentation mit Swagger

Alle Endpunkte werden automatisch durch **Swashbuckle** (Swagger) dokumentiert und sind unter `/swagger` erreichbar. Im Entwicklungsmodus können Endpunkte direkt aus der Swagger-UI heraus getestet werden. Für gesicherte Endpunkte kann dort ein Bearer-Token hinterlegt werden. Abbildung 5.1 zeigt die Swagger-UI mit den verfügbaren Endpunkten.

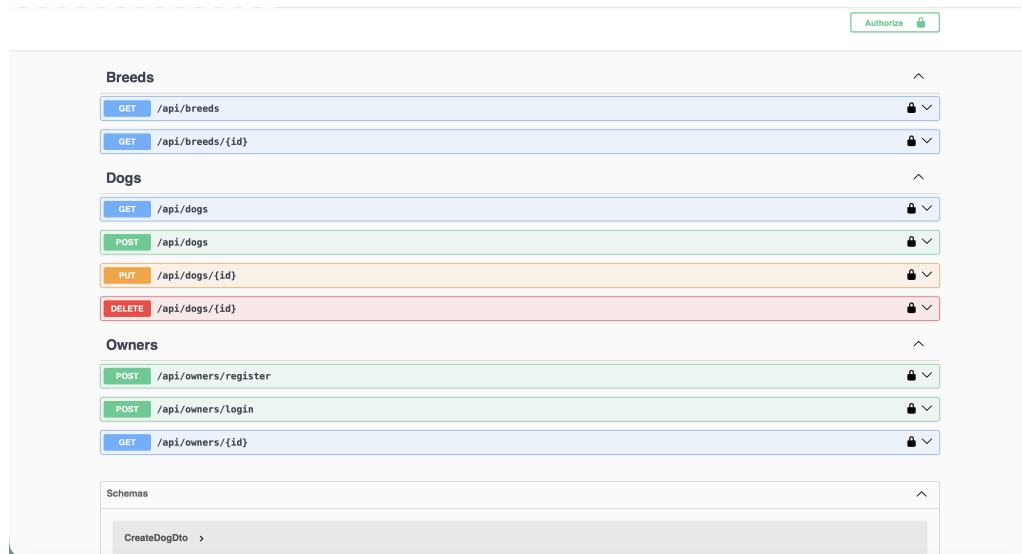


Abbildung 5.1: Swagger-UI des DoggoGO-Backends

Kapitel 6

iOS-Applikation

6.1 Architektur der iOS-App

Die iOS-Applikation wurde mit **Swift** und **SwiftUI** entwickelt und folgt dem **MVVM-Muster** (Model-View-ViewModel). Diese Architektur trennt die UI-Darstellung (View) von der Geschäftslogik und dem Zustand (ViewModel) und erleichtert Testbarkeit und Wartbarkeit.

Model Entspricht den Datenstrukturen, die der Backend-API entsprechen (**Dog**, **Owner**, **Breed**, **LegalRestrictions**).

ViewModel Verwaltet den Zustand der UI mit **@Published**-Eigenschaften und kommuniziert mit dem Backend über HTTP. Durch das **@StateObject**-Macro wird der Lebenszyklus an die View gebunden.

View Deklarative SwiftUI-Views lesen den Zustand aus dem ViewModel und reagieren automatisch auf Änderungen.

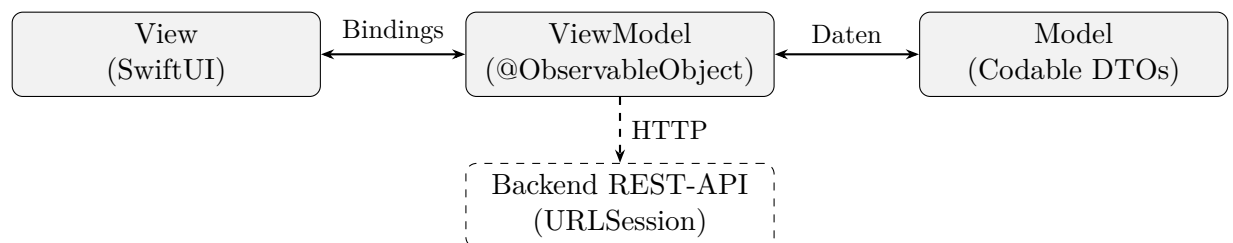


Abbildung 6.1: MVVM-Architektur der DoggoGO iOS-App

6.2 Projektstruktur

Das Xcode-Projekt ist in folgende Gruppen gegliedert:

- `Models/` – Codable-Strukturen für API-DTOs
- `ViewModels/` – `ObservableObject`-Klassen mit Geschäftslogik
- `Views/` – SwiftUI-Views (Login, Map, DogList, BreedPicker, etc.)
- `Services/` – `APIService` (HTTP-Kommunikation), `AuthService` (Keychain-Zugriff)
- `Assets.xcassets` – App-Icons, Farbschemata

6.3 MapKit-Integration

Die Kartenansicht basiert auf **MapKit**, Apples nativer Kartenbibliothek. In SwiftUI wird eine Karte mit `Map(coordinateRegion:)` eingebunden. Der aktuelle Standort der Nutzerin / des Nutzers wird über `CLLocationManager` ermittelt und als Region übergeben.

```
1 import SwiftUI
2 import MapKit
3
4 struct MapView: View {
5     @StateObject private var locationManager = LocationManager()
6     @State private var region = MKCoordinateRegion(
7         center: CLLocationCoordinate2D(
8             latitude: 48.2692, longitude: 14.2956), // Leonding
9         span: MKCoordinateSpan(
10             latitudeDelta: 0.05, longitudeDelta: 0.05)
11     )
12
13     var body: some View {
14         Map(coordinateRegion: $region,
15             showsUserLocation: true,
16             annotationItems: locationManager.pois) { poi in
17             MapMarker(coordinate: poi.coordinate,
18                 tint: poi.color)
19         }
20         .ignoresSafeArea()
21         .onReceive(locationManager.$userLocation) { loc in
22             guard let loc else { return }
23             region.center = loc.coordinate
24         }
25     }
26 }
```

Listing 6.1: Kartenansicht in SwiftUI (vereinfacht)

Für die Darstellung von Points of Interest (Freilaufwiesen, Sackerlspender, Hundekot-Mülleimer) werden `MapMarker`-Annotationen mit farblicher Codierung pro Kategorie genutzt.

6.4 Standortdienste und Berechtigungen

Die Standortermittlung wird über `CLLocationManager` realisiert. Gemäß DSGVO-Anforderung (NF-03) darf der Standort nur mit expliziter Nutzerinnen-/Nutzerzustimmung erfasst werden. Die App fragt beim ersten Start die Berechtigung `requestWhenInUseAuthorization()` ab. Die `Info.plist` enthält die Pflichteinträge `NSLocationWhenInUseUsageDescription` mit einer verständlichen Begründung.

```
1 class LocationManager: NSObject, ObservableObject,
  CLLocationManagerDelegate {
2   private let manager = CLLocationManager()
3   @Published var userLocation: CLLocation?
4
5   override init() {
6       super.init()
7       manager.delegate = self
8       manager.desiredAccuracy = kCLLocationAccuracyBest
9       manager.requestWhenInUseAuthorization()
10      manager.startUpdatingLocation()
11  }
12
13  func locationManager(_ manager: CLLocationManager,
14                      didUpdateLocations locations: [CLLocation]
15  ) {
16      userLocation = locations.last
17  }
```

Listing 6.2: LocationManager (Auszug)

6.5 Authentifizierung in der iOS-App

6.5.1 Login und Registrierung

Die Anmeldeansicht (`LoginView`) enthält Eingabefelder für E-Mail und Passwort. Nach dem Tippen auf „Anmelden“ wird `POST /api/owners/login` mit den Credentials aufgerufen. Bei Erfolg liefert das Backend ein JWT zurück:

```
1 func login(email: String, password: String) async throws {
2     let body = LoginRequest(email: email, password: password)
3     let response: AuthResponse = try await apiService.post(
4         endpoint: "/api/owners/login", body: body)
5     AuthService.shared.saveToken(response.token)
6     isAuthenticated = true
7 }
```

Listing 6.3: Login-Anfrage (Auszug)

6.5.2 Token-Persistenz im Keychain

Das JWT wird sicher im iOS Keychain gespeichert, nicht in `UserDefaults`, da der Keychain verschlüsselt und vor anderen Apps geschützt ist. Der `AuthService` kapselt alle Keychain-Operationen:

```
1 class AuthService {
2     static let shared = AuthService()
3     private let tokenKey = "doggo_jwt_token"
4
5     func saveToken(_ token: String) {
6         let data = Data(token.utf8)
7         let query: [String: Any] = [
8             kSecClass as String: kSecClassGenericPassword,
9             kSecAttrAccount as String: tokenKey,
10            kSecValueData as String: data
11        ]
12        SecItemDelete(query as CFDictionary)
13        SecItemAdd(query as CFDictionary, nil)
14    }
15
16    func getToken() -> String? {
17        let query: [String: Any] = [
18            kSecClass as String: kSecClassGenericPassword,
19            kSecAttrAccount as String: tokenKey,
20            kSecReturnData as String: true
21        ]
22        var item: CftypeRef?
23        guard SecItemCopyMatching(query as CFDictionary, &item) ==
noErr,
24            let data = item as? Data else { return nil }
25        return String(data: data, encoding: .utf8)
26    }
27 }
```

Listing 6.4: AuthService – Keychain-Speicherung (Auszug)

Beim App-Start prüft das Root-ViewModel, ob ein Token im Keychain vorhanden ist, und überspringt in diesem Fall die Anmeldeseite direkt zur Kartenansicht.

6.6 Hundeprofilverwaltung

Die `DogListView` listet alle Hunde der angemeldeten Nutzerin / des angemeldeten Nutzers auf und ruft dazu `GET /api/dogs` mit dem Bearer-Token auf. Über einen `NavigationLink` gelangt man zur Detailansicht oder zum Formular zum Anlegen / Bearbeiten eines Profils. Die Rassenauswahl erfolgt über einen `Picker`, der die von `GET /api/breeds` geladene Liste anzeigt.


```
1 struct DogListView: View {
2     @StateObject private var viewModel = DogListViewModel()
3
4     var body: some View {
5         NavigationView {
6             List(viewModel.dogs) { dog in
7                 NavigationLink(destination: DogDetailView(dog: dog)
8             ) {
9                 VStack(alignment: .leading) {
10                     Text(dog.name).font(.headline)
11                     Text("\(dog.breed.breedName), \(dog.age)
12                     Jahre")
13                     .font(.subheadline).foregroundColor(.
14                     secondary)
15                 }
16             }
17             .navigationTitle("Meine Hunde")
18             .toolbar {
19                 ToolbarItem(placement: .navigationBarTrailing) {
20                     NavigationLink("Hinzufuegen",
21                     destination: AddDogView())
22                 }
23             }
24             .task { await viewModel.loadDogs() }
25 }
```

Listing 6.5: DogListView – Übersicht der Hundepprofile (Auszug)

6.7 Anzeige gesetzlicher Vorschriften

Nach der Rassenauswahl ruft die App GET `/api/breeds/{id}` ab und zeigt die `LegalRestrictions` in einer strukturierten Ansicht an. Rassen der Kategorie `spezielle_hunderasse` werden farblich hervorgehoben (rotes Label), um die Nutzerin / den Nutzer auf besondere Pflichten aufmerksam zu machen:

```
1 struct LegalRestrictionsView: View {
2     let restrictions: LegalRestrictions
3
4     var body: some View {
5         VStack(alignment: .leading, spacing: 8) {
6             Text("Rechtliche Vorgaben")
7                 .font(.title2).bold()
8             if restrictions.category == "spezielle_hunderasse" {
9                 Text("Spezielle Hunderasse!")
10                    .foregroundColor(.red).bold()
11             }
12             ForEach(restrictions.rules, id: \.self) { rule in
13                 Label(rule, systemImage: "exclamationmark.circle")
14                     .font(.callout)
15             }
16         }
17         .padding()
18     }
19 }
```

Listing 6.6: LegalRestrictionsView – Anzeige der Vorschriften

6.8 HTTP-Kommunikation mit dem Backend

Alle Netzwerkanfragen werden über einen zentralen `APIService` abgewickelt, der `URLSession` kapselt. Das Bearer-Token wird in jedem Request automatisch als `Authorization-Header` eingefügt:

6.9 Fehlerbehandlung und Ladezustände

Eine robuste iOS-App muss mit Netzwerkfehlern, ungültigen Tokens und unerwarteten Serverantworten umgehen. In der DoggoGO-App wird Fehlerbehandlung konsequent im ViewModel-Layer zentralisiert – die View selbst kennt keine Netzwerklogik.

6.9.1 Fehlerdefinition

Alle möglichen Fehlerszenarien werden in einem `APIError-Enum` typisiert:

```

1 func get<T: Decodable>(endpoint: String) async throws -> T {
2     guard let url = URL(string: baseUrl + endpoint) else {
3         throw APIError.invalidURL
4     }
5     var request = URLRequest(url: url)
6     if let token = AuthService.shared.getToken() {
7         request.setValue("Bearer \(token)",
8             forHTTPHeaderField: "Authorization")
9     }
10    let (data, response) = try await URLSession.shared.data(for:
11    request)
12    guard let http = response as? HTTPURLResponse,
13        (200..<300).contains(http.statusCode) else {
14        throw APIError.serverError
15    }
16    return try JSONDecoder().decode(T.self, from: data)
17 }

```

Listing 6.7: APIService – Authentifizierter Request (Auszug)

```

1 enum APIError: LocalizedError {
2     case invalidURL
3     case unauthorized          // 401 - JWT abgelaufen oder ungültig
4     case notFound              // 404 - Ressource nicht vorhanden
5     case serverError           // 5xx - Backend-Fehler
6     case decodingError         // JSON-Parsing fehlgeschlagen
7     case noConnection          // Netzwerk nicht erreichbar
8
9     var errorDescription: String? {
10        switch self {
11            case .invalidURL:      return "Ungültige URL."
12            case .unauthorized:    return "Sitzung abgelaufen. Bitte
13            erneut anmelden."
14            case .notFound:        return "Ressource nicht gefunden."
15            case .serverError:     return "Serverfehler. Bitte später
16            versuchen."
17            case .decodingError:   return "Antwort konnte nicht
18            verarbeitet werden."
19            case .noConnection:    return "Keine Internetverbindung."
20        }
21    }
22 }

```

Listing 6.8: APIError – Typisierte Fehlerdefinition

Das Protokoll `LocalizedError` stellt sicher, dass SwiftUI und das Betriebssystem die Fehlerbeschreibung automatisch für Alert-Dialoge verwenden können.

6.9.2 Ladezustand und Fehleranzeige im ViewModel

Jedes ViewModel hält zwei Zustandsvariablen: `isLoading` blendet während einer laufenden Netzwerkanfrage eine Ladeindikator-Ansicht ein, `errorMessage` speichert eine lesbare Fehlerbeschreibung für den Alert-Dialog:

```
1 class DogListViewModel: ObservableObject {
2     @Published var dogs: [Dog] = []
3     @Published var isLoading = false
4     @Published var errorMessage: String?
5
6     func loadDogs() async {
7         isLoading = true
8         defer { isLoading = false }
9         do {
10             dogs = try await APIService.shared.get(endpoint: "/api/
dogs")
11         } catch APIError.unauthorized {
12             errorMessage = APIError.unauthorized.errorDescription
13             AuthService.shared.deleteToken() // abgelaufenes
Token löschen
14         } catch {
15             errorMessage = error.localizedDescription
16         }
17     }
18 }
```

Listing 6.9: ViewModel – Ladezustand und Fehlerbehandlung

Das `defer`-Statement stellt sicher, dass `isLoading` immer auf `false` zurückgesetzt wird – auch wenn die Anfrage mit einem Fehler endet. Bei einem 401 `Unauthorized` wird das gespeicherte JWT automatisch gelöscht, sodass die Nutzerin / der Nutzer zur Anmeldeseite weitergeleitet wird.

6.9.3 Alert-Anzeige in der View

Die View abonniert `errorMessage` über ein `.alert`-Modifier und zeigt bei gesetztem Wert automatisch einen Dialog an:

```
1 .alert("Fehler", isPresented: Binding(
2     get: { viewModel.errorMessage != nil },
3     set: { if !$0 { viewModel.errorMessage = nil } }
4 )) {
5     Button("OK", role: .cancel) { }
6 } message: {
```

```
7      Text(viewModel.errorMessage ?? "")
8  }
9  .overlay {
10      if viewModel.isLoading {
11          ProgressView("Laden ...")
12              .padding()
13              .background(.regularMaterial, in: RoundedRectangle(
14                  cornerRadius: 8))
15      }
16  }
```

Listing 6.10: SwiftUI – Alert fuer Fehlermeldungen

Dieser Ansatz hält die View vollständig frei von Fehlerlogik: Sie bindet lediglich den Zustand des ViewModels und delegiert jede Entscheidung dorthin.

Kapitel 7

Testing und Deployment

7.1 Teststrategie

Eine systematische Teststrategie stellt sicher, dass das Backend und die iOS-App korrekt funktionieren und die definierten Anforderungen erfüllen. Für DoggoGO wurden drei Testebenen eingesetzt:

1. **Manuelle API-Tests** über Swagger UI und Postman
2. **Unit-Tests** für einzelne Backend-Komponenten (xUnit)
3. **Integrationstests** für das Zusammenspiel zwischen iOS-App und Backend

7.2 Backend-Tests

7.2.1 Manuelle Tests mit Swagger und Postman

Während der Entwicklung wurden alle API-Endpunkte zunächst manuell über die **Swagger-UI** getestet. Dies erlaubt schnelles Feedback ohne Testrahmenaufwand: Registrierung, Anmeldung, CRUD-Operationen für Hunde sowie das Abrufen der Rassenliste wurden iterativ überprüft. Für komplexere Szenarien (z. B. Fehlerfälle, ungültige Tokens) wurde **Postman** mit einer gespeicherten Collection genutzt, die die wichtigsten Endpunkte mit vorbereiteten Request-Bodies enthält.

Tabelle 7.1 listet die wichtigsten getesteten Szenarien auf:

Tabelle 7.1: Getestete API-Szenarien

Szenario	Endpunkt	Erwartetes Ergebnis	Status
Registrierung (gültig)	POST /owners/register	201 Created, Owner-Objekt	OK
Registrierung (doppelt)	POST /owners/register	409 Conflict (doppelte Email)	OK
Anmeldung (gültig)	POST /owners/login	200 OK, JWT	OK
Anmeldung (Falsches PW)	POST /owners/login	401 Unauthorized	OK
Hund anlegen (auth.)	POST /api/dogs	201 Created, Dog-Objekt	OK
Hund anlegen (unauth.)	POST /api/dogs	401 Unauthorized	OK
Hund löschen (Cascade)	DELETE /owners/{id}	204 + Hunde gelöscht	OK
Rassen abrufen	GET /api/breeds	200 OK, Rassenliste mit Regeln	OK

7.2.2 Unit-Tests mit xUnit

Für die Passwort-Hashing-Logik und die JWT-Erzeugung wurden Unit-Tests mit **xUnit** implementiert. Listing 7.1 zeigt einen exemplarischen Test für den Passwort-Hash-Vergleich:

7.3 iOS-Tests

Die iOS-App wurde mit **XCTest** getestet. Aufgrund des Umfangs der Diplomarbeit konzentrierten sich die Tests auf das **AuthService**-Modul (Keychain-Operationen) und die **APIService**-Klasse (URL-Konstruktion, Fehlerbehandlung):

7.4 Integrationstests

Integrationstests überprüfen das Ende-zu-Ende-Verhalten des Gesamtsystems: Die iOS-App wird im Simulator gestartet, und die Benutzerabläufe (Registrierung → Anmeldung → Hundeprofi anlegen → Rasse mit Rechtsvorschriften abrufen) werden manuell durchgeführt. Die HTTP-Anfragen wurden dabei mit dem Netzwerk-Inspector in Xcode beobachtet, um korrekte Request/Response-Zyklen zu verifizieren.

```
1 public class PasswordHashTests
2 {
3     [Fact]
4     public void VerifyPassword_CorrectPassword_ReturnsTrue()
5     {
6         // Arrange
7         using var hmac = new HMACSHA512();
8         var salt = hmac.Key;
9         var hash = hmac.ComputeHash(Encoding.UTF8.GetBytes("
10 Test1234!"));
11
12         // Act
13         using var hmac2 = new HMACSHA512(salt);
14         var verifyHash = hmac2.ComputeHash(
15             Encoding.UTF8.GetBytes("Test1234!"));
16
17         // Assert
18         Assert.Equal(hash, verifyHash);
19     }
20
21     [Fact]
22     public void VerifyPassword_WrongPassword_ReturnsFalse()
23     {
24         using var hmac = new HMACSHA512();
25         var salt = hmac.Key;
26         var hash = hmac.ComputeHash(Encoding.UTF8.GetBytes("
27 Test1234!"));
28
29         using var hmac2 = new HMACSHA512(salt);
30         var verifyHash = hmac2.ComputeHash(
31             Encoding.UTF8.GetBytes("WrongPassword"));
32
33         Assert.NotEqual(hash, verifyHash);
34     }
35 }
```

Listing 7.1: xUnit-Test – Passwort-Hash-Verifikation


```
1 class AuthServiceTests: XCTestCase {
2
3     func testSaveAndRetrieveToken() {
4         let service = AuthService.shared
5         let testToken = "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.test"
6
7         service.saveToken(testToken)
8         let retrieved = service.getToken()
9
10        XCTAssertEqual(testToken, retrieved)
11    }
12
13    func testDeleteToken() {
14        let service = AuthService.shared
15        service.saveToken("some.token.value")
16        service.deleteToken()
17
18        XCTAssertNil(service.getToken())
19    }
20 }
```

Listing 7.2: XCTest – AuthService Keychain-Test

7.5 Deployment-Verifikation

Nach einem `docker compose up -build` wurden folgende Punkte geprüft:

- PostgreSQL-Container läuft und ist über Port 5432 erreichbar.
- Backend-Container startet, Datenbankmigrationen werden ohne Fehler angewendet.
- Breed-Seeding protokolliert „*Breeds seeded successfully*“.
- Swagger-UI ist unter `http://localhost:8080/swagger` erreichbar.
- Registrierung und Anmeldung über Swagger-UI funktionieren korrekt.

Kapitel 8

Zusammenfassung

8.1 Erreichte Ziele

Im Rahmen dieser Diplomarbeit wurden die beiden zentralen Komponenten von DoggoGO – Backend und iOS-Applikation – konzipiert und implementiert.

Das **Backend** auf Basis von ASP.NET Core 8 stellt eine vollständige RESTful API mit Registrierung, JWT-Authentifizierung, Hundeprofilverwaltung und Rassendatenverwaltung bereit. Die Datenhaltung erfolgt über PostgreSQL mit EF Core 9; das automatische Breed-Seeding aus einer JSON-Datei macht die Rassen-Datenbank sofort nutzbar. Das gesamte System ist mittels Docker Compose containerisiert und damit plattformunabhängig deploybar.

Die **iOS-App** bietet eine kartenbasierte Darstellung des Benutzerstandorts (MapKit), sichere JWT-Authentifizierung mit Keychain-Persistenz, Hundeprofilverwaltung sowie die Anzeige rassenspezifischer Rechtsvorschriften. Die MVVM-Architektur mit SwiftUI ermöglicht eine klare Trennung von Darstellung und Logik.

Alle in Kapitel 2 definierten funktionalen Anforderungen (FA-B01 bis FA-B08 und FA-I01 bis FA-I08) wurden erfüllt. Die nicht-funktionalen Anforderungen bezüglich Sicherheit (NF-01, NF-02), Datenschutz (NF-03) und Portierbarkeit (NF-06) wurden ebenfalls umgesetzt.

8.2 Suboptimale Entscheidungen

Im Rückblick lassen sich einige Entscheidungen identifizieren, die bei einer Neuentwicklung anders getroffen würden:

Doppelter Foreign-Key in der Initialmigration Die automatisch generierte EF Core-Migration `InitDatabase` enthielt einen redundanten Foreign-Key `LegalRestrictionsId1` in der `Breeds`-Tabelle. Dieser entstand durch eine nicht klar definierte `WithMany()`-Konfiguration und musste manuell aus der Migration entfernt werden. Eine sorgfältigere Konfiguration in `OnModelCreating` hätte diesen Fehler vermieden.

Fehlendes Controller-Layer Das Backend wurde zunächst ohne dedizierte Controller-Klassen begonnen; Endpunkte wurden direkt in `Program.cs` als Minimal API definiert. Erst im Verlauf des Projekts wurde auf den klassischen Controller-Ansatz umgestellt, was zu einer Umstrukturierung des Codes führte. Ein frühzeitiger Entschluss für einen der beiden Ansätze hätte Zeit gespart.

Fehlende Input-Validierung In der aktuellen Version werden Eingaben (z. B. beim Anlegen eines Hundes) nicht serverseitig mit `DataAnnotations` oder einem Validierungsframework validiert. Ungültige Werte (leerer Name, negatives Alter) könnten so in die Datenbank gelangen. Für eine produktive Anwendung ist eine umfassende Validierungsschicht zwingend erforderlich.

iOS-Simulator statt echtes Gerät Die iOS-App wurde ausschließlich im Xcode-Simulator getestet. GPS-Standortdaten wurden simuliert. Tests auf einem physischen Gerät hätten möglicherweise Probleme mit der Standortgenauigkeit oder Keychain-Berechtigungen aufgedeckt.

8.3 Einschränkungen der Lösung

- **Keine echten Geo-Daten:** Freilaufwiesen, Sackerlspender und Hundekot-Mülleimer sind im aktuellen Stand nicht in der Datenbank enthalten. Die Kartendarstellung zeigt ausschließlich den Benutzerstandort. Die Entitäten und API-Endpunkte für Standortdaten wurden geplant, konnten im Rahmen der Diplomarbeit aber nicht vollständig implementiert werden.
- **Nur iOS, kein Android:** Eine Android-App ist nicht Teil des Projekts. Die API-Architektur ist jedoch so gestaltet, dass ein Android-Client ohne Backend-Änderungen ergänzt werden könnte.
- **Keine Produktionsumgebung:** Das System wurde für Entwicklungs- und Demonstrationszwecke konfiguriert. Für einen produktiven Betrieb wären HTTPS-Terminierung, Secrets-Management (z. B. Vault) und ein Reverse-Proxy (Nginx)

notwendig.

- **Regionale Begrenzung:** Die Rechtsvorschriften sind derzeit auf österreichisches Recht (Wiener Hundegesetz) beschränkt. Eine Ausweitung auf andere Bundesländer oder Länder erfordert weitere Datenerhebung und -pflege.

8.4 Erweiterungsmöglichkeiten

Das System bietet eine solide Basis für zahlreiche Erweiterungen:

Standort-Daten (DOG3–DOG5, DOG17) Import von Open-Data-Geodaten (z. B. der Stadt Wien) für Freilaufwiesen, Sackerlspender und Hundewaschstationen. Neue `Location`-Entität im Backend, REST-Endpunkte für Geo-Abfragen.

Geofencing-Benachrichtigungen (DOG11) Push-Benachrichtigungen via APNs, wenn die Nutzerin / der Nutzer eine Zone mit besonderen Vorschriften betritt. Serverseitig über einen Background-Worker realisierbar.

Routenplanung (DOG14) Integration von MapKit-Routing zur Navigation zur nächsten Freilaufwiese.

Favoriten (DOG15) Persistenz von Lieblings-Standorten in der Datenbank.

Android-App Da die REST-API plattformunabhängig ist, kann ein Android-Client (Kotlin / Jetpack Compose) ohne Backend-Änderungen ergänzt werden.

Community-Features Nutzerbewertungen und Erfahrungsberichte zu Standorten (neue Entitäten `Review`, `Report`).

Anhang A

API-Endpunkte im Detail

Dieser Anhang listet alle implementierten und geplanten REST-Endpunkte der DoggoGO-Backend-API mit Request/Response-Schemata auf. Die vollständige und interaktive Dokumentation ist über die Swagger-UI unter `/swagger` erreichbar.

Authentifizierung

POST `/api/owners/register`

- **Beschreibung:** Registriert eine neue Nutzerin / einen neuen Nutzer.
- **Request-Body (JSON):**

```
1 {  
2   "name": "Max Muster",  
3   "email": "max@example.com",  
4   "password": "SecurePass123!",  
5   "phoneNumber": "+43 123 456789"  
6 }
```

- **Response 201:** Owner-Objekt (ohne Passwort-Felder)
- **Response 409:** E-Mail bereits vergeben

POST `/api/owners/login`

- **Beschreibung:** Meldet eine Nutzerin / einen Nutzer an und gibt ein JWT zurück.
- **Request-Body (JSON):**

```
1 {  
2   "email": "max@example.com",  
3   "password": "SecurePass123!"  
4 }
```

- **Response 200:** {"token": "eyJ..."}
- **Response 401:** Ungültige Anmeldedaten

Hunde (Dogs)

GET /api/dogs (*Authentifizierung erforderlich*)

Gibt alle Hundepprofile der aktuellen Nutzerin / des aktuellen Nutzers zurück.

POST /api/dogs (*Authentifizierung erforderlich*)

Legt ein neues Hundeprofil an. Request-Body (JSON):

```
1 {  
2   "name": "Bello",  
3   "age": 3,  
4   "breedId": "a1b2c3d4-..."  
5 }
```

PUT /api/dogs/{id} (*Authentifizierung erforderlich*)

Aktualisiert ein bestehendes Hundeprofil.

DELETE /api/dogs/{id} (*Authentifizierung erforderlich*)

Löscht ein Hundeprofil.

Hunderassen (Breeds)

GET /api/breeds

Gibt alle Hunderassen inklusive `LegalRestrictions` zurück.

GET /api/breeds/{id}

Gibt eine einzelne Rasse mit vollständigen Rechtsvorschriften zurück.

Anhang B

Dokumentation der KI-Nutzung

Gemäß den Richtlinien der HTL Leonding müssen alle im Rahmen dieser Diplomarbeit verwendeten KI-generierten Inhalte dokumentiert werden. Die folgende Tabelle listet alle Einsätze auf, bei denen ein KI-Tool zur Unterstützung herangezogen wurde.

Datum	KI-Tool	Verwendungszweck / Prompt (Zusammenfassung)	Kapitel / Datei
20.02.2026	Claude (Anthropic)	Strukturvorschlag für Kapitel Systemarchitektur; Überarbeitung auf Basis eigener Inhalte und Projektcode	Kap. 3
25.08.2025	Claude (Anthropic)	Vorlage für JWT-Authentifizierungslogik; angepasst an eigene Modelle und Konfiguration	Kap. 5
22.01.2026	Claude (Anthropic)	Entwurf der SwiftUI-Listings; geprüft und an eigene Projektstruktur angepasst	Kap. 6
<i>Alle KI-generierten Inhalte wurden kritisch geprüft, inhaltlich angepasst und auf ihre Korrektheit hin verifiziert. Die endgültige Verantwortung für den Inhalt liegt beim Autor dieser Arbeit.</i>			